

IFT6163 HW3: Making a Generalist Robotics Policy with Reinforcement Learning

Samuel

March 2026

Task: LIBERO Spatial, Task 9 — Place Akita Black Bowl onto Plate

Observation: 13-dim privileged state [eef_pos(3), eef_quat_xyz(3), gripper(1), bowl_rel(3), plate_rel(3)]

Action: 7-dim [Δ pos(3), Δ ori(3), gripper(1)]

1 Part 1: Dense Policy with PPO

1.1 Reward Function

```
bowl_rel      = obs[7:10]                # bowl position relative to EEF
plate_rel     = obs[10:13]               # plate position relative to EEF
bowl_pos_abs  = eef_pos + bowl_rel
plate_pos_abs = eef_pos + plate_rel
bowl_to_plate = bowl_pos_abs - plate_pos_abs # bowl -> plate vector

reward_gripper_bowl = -np.linalg.norm(bowl_rel)
reward_bowl_plate   = -np.linalg.norm(bowl_to_plate)
reward = (reward_gripper_bowl + reward_bowl_plate) * 0.1
```

The reward is a sum of two Euclidean distance penalties:

1. $r_1 = -\|bowl_rel\|$: encourages the end-effector (EEF) to approach the bowl.
2. $r_2 = -\|bowl_pos - plate_pos\|$: encourages the bowl to be close to the plate.

Both are scaled by 0.1 to keep per-step rewards in $[-0.1, 0)$, preventing excessively large value-function targets.

Expected optimal behavior. An optimal policy sets both distances to zero: first grasping the bowl ($r_1 \rightarrow 0$) then placing it on the plate ($r_2 \rightarrow 0$), reaching total reward ≈ 0 per step.

Limitation. The reward does not explicitly require grasping. A policy that hovers the EEF near the bowl without closing the gripper achieves small r_1 while r_2 remains large, trapping it in a hovering local optimum. This was observed in practice (see failure case below).

1.2 PPO Implementation

Policy network: 3-layer MLP (256 hidden, ReLU), Gaussian head with learned diagonal $\log \sigma$ per action dimension.

Value network: Separate 3-layer MLP (256 hidden). Networks are not shared.

Advantage estimation: Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

with $\gamma = 0.99$, $\lambda = 0.95$. Advantages are normalized per rollout to zero mean and unit variance. Returns $\hat{R}_t = \hat{A}_t + V(s_t)$ are *not* normalized so that the value function learns the absolute return scale.

Clipped surrogate:

$$\mathcal{L}_\pi = -\mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 \pm \varepsilon) \hat{A}_t \right) \right] - \alpha H[\pi_\theta]$$

with $\varepsilon = 0.2$, $\alpha = 0.01$ (entropy coefficient).

Value loss: $\mathcal{L}_V = \mathbb{E}[(V_\phi(s_t) - \hat{R}_t)^2]$.

Hyperparameters: rollout_length=2048, ppo_epochs=10, minibatch=256, lr=3e-4, grad_clip=0.5.

1.3 Results

Seed	Final Episode Return (200k steps)
0	−38.4
1	−54.9
2	−52.1
Mean $\pm$ std	−48.5 $\pm$ 8.9

Table 1: Dense PPO results across three seeds, LIBERO Spatial Task 9.

All three seeds show improvement from approximately −100 (at 10k steps) to −40 to −55 (at 200k steps), confirming that the dense policy learns to reduce both distance terms over training.

Failure case. Seed 1 converges to a hovering policy: the EEf moves close to the bowl, satisfying $r_1 \approx 0$, but the gripper never closes. The bowl is never lifted and r_2 stays large. The reward function offers no gradient signal to explicitly reward gripper closure, so this local optimum is stable. A potential fix would be adding a binary contact reward upon successful grasp.

2 Part 2: Fine-Tuning a Transformer Policy with RL

2.1 Architecture and Initialization

The transformer policy is the GRP model from HW1 (`hw3/grp_model.py`), loaded from `hw3/miniGRP_Model/miniGRP`. The model was pre-trained on LIBERO demonstrations via behavior cloning from image observations.

Key model attributes (verified at load time):

- `class_tokens`: shape (1, 1, 128) — learnable CLS token, embedding dim $d = 128$
- `lin_map_pose`: `Linear(7, 128)` — maps pose input to embedding space
- `blocks`: stacked transformer self-attention blocks

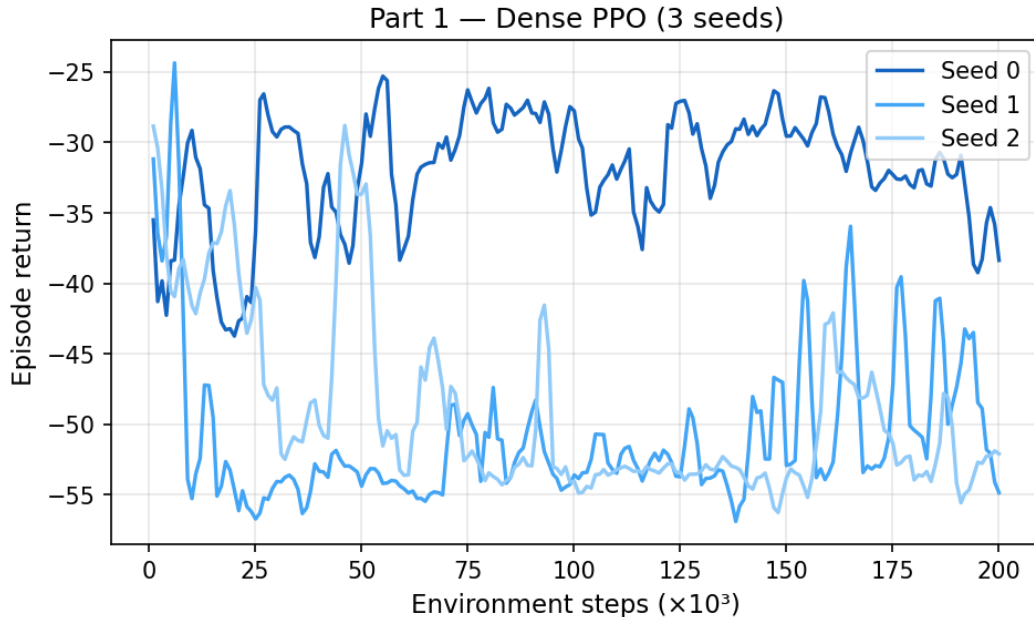


Figure 1: Dense PPO episode return over training for three seeds (LIBERO Spatial Task 9). Seed 0 (darkest) achieves the best final return of -38.4 ; seeds 1 and 2 converge to -54.9 and -52.1 respectively. The shaded band shows the inter-seed range.

- `mlp`: output projection head

Adaptation for RL. Since the HW1 model expects a 7-dim pose input but the RL environment provides a 13-dim state, a linear adapter `pose_adapter: Linear(13, 7)` (randomly initialized) is prepended. A learned diagonal log-std parameter `rl_log_std` $\in \mathbb{R}^7$ is added for the Gaussian action head. The value function is a **separate** 3-layer MLP, not sharing weights with the transformer.

The forward pass for action generation:

$$\text{obs}(13) \xrightarrow{\text{pose_adapter}} (7) \xrightarrow{\text{lin_map_pose}} (128) \rightarrow [\text{CLS} \| s_t] \xrightarrow{+\text{pos_emb}} \text{blocks} \rightarrow \text{ln.f} \rightarrow \text{mlp} \rightarrow \mu_a(7)$$

$$a \sim \mathcal{N}(\mu_a, e^{\text{rl_log_std}})$$

2.2 (a) PPO Fine-Tuning

Experiment	Initial Return	Final Return (200k steps)	Trend
Transformer PPO lr=1e-5	-29.9	-29.9	Flat
Transformer PPO lr=3e-4	≈ -30	-39.4	Degraded
Dense PPO seed0 (Part 1)	≈ -100	-38.4	Improved

Table 2: Transformer PPO vs. Dense PPO comparison.

Observations. At `lr=1e-5` the transformer policy does not improve: the return stays flat at ≈ -30 throughout 200k steps. At `lr=3e-4`, the pre-trained initialization degrades from -30 to -39.4 , suggesting that the larger learning rate progressively overwrites the task-relevant pre-trained features.

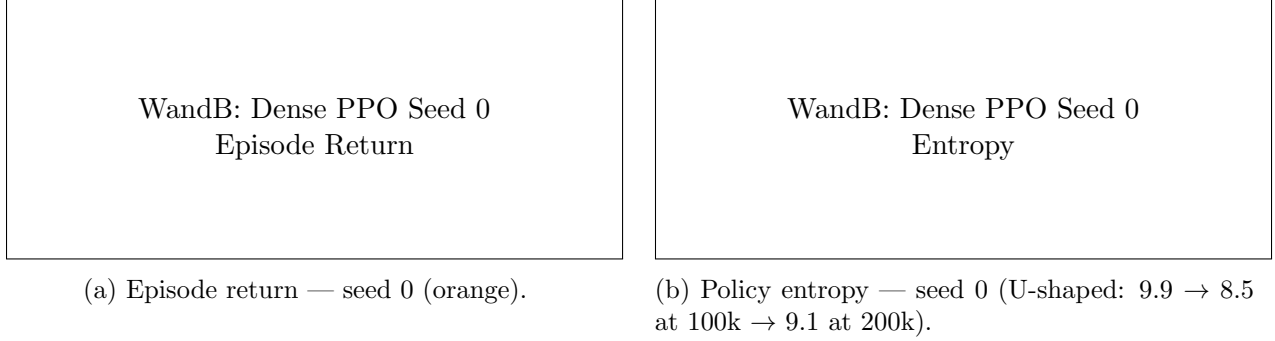


Figure 2: WandB training metrics for Dense PPO seed 0. The entropy curve shows the policy first specialising (decreasing entropy) then partially re-exploring, consistent with the oscillatory return curve.

Transfer analysis. Components that transferred *poorly*:

- `lin_map_pose`: pre-trained for 7-dim joint pose from demonstration data. The added `pose_adapter(13 $\rightarrow$ 7)` is randomly initialized and creates a distribution mismatch at the first layer.
- `mlp` output head: pre-trained to predict actions from image-level CLS features. The action semantics differ in the RL setting.

Components that transferred *well*:

- The transformer blocks provide a useful inductive bias for sequential token interaction.
- The pre-trained checkpoint yields a reasonable starting policy (≈ -30), outperforming the dense policy’s initial performance (≈ -100) and saving tens of thousands of environment steps.

Comparison with dense PPO. Dense PPO (seed0) improves from -100 to -38.4 at 200k steps. Transformer PPO stays at -29.9 . The transformer has a better final return by this comparison, but that is entirely due to the pre-trained initialization — no RL improvement occurred. Dense PPO is more sample-efficient in the sense that it actually learns: transformer PPO achieves zero RL sample efficiency in terms of improvement over initialization.

2.3 (b/c) GRPO with Ground-Truth Resets

2.3.1 How sampled trajectories differ

From a fixed initial state s_0 (obtained via `env.env.sim.get_state()` and restored with `env.env.sim.set_state(i + env.env.sim.forward())`), a group of `group_size=8` trajectories is sampled by running the stochastic policy from s_0 . Because the Gaussian action distribution has nonzero variance, each trajectory samples different actions at step 0 and diverges subsequently through the environment dynamics. Over a 600-step episode, small action differences compound into substantially different total returns.

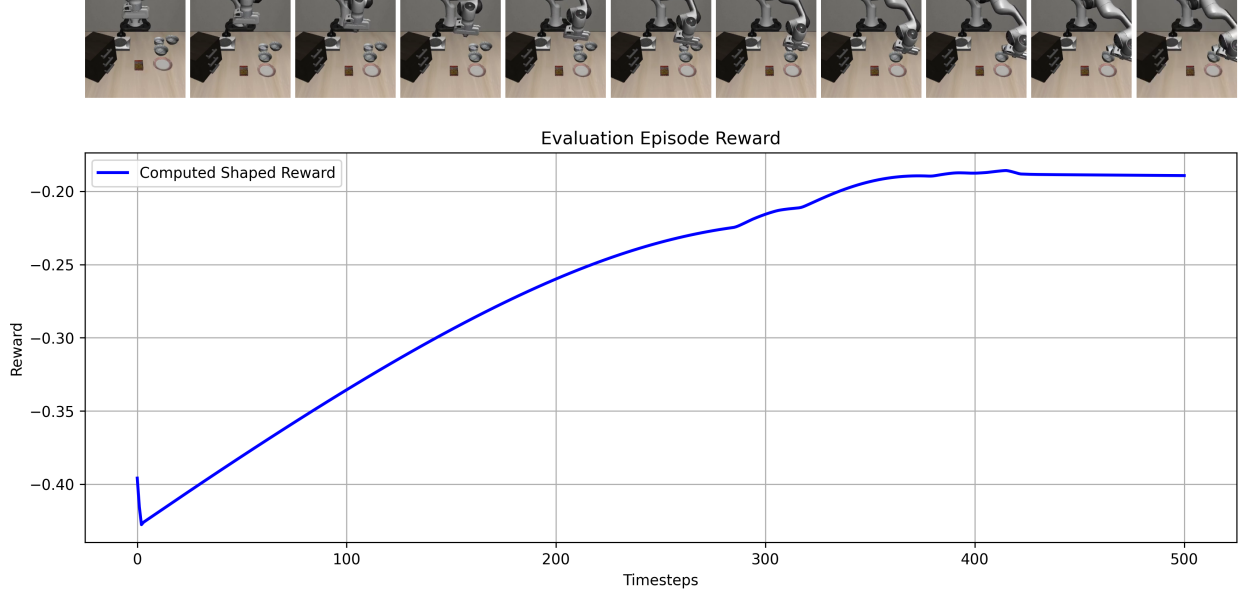


Figure 3: Required Part 2 composite figure. **Top row:** sampled frames from the best-performing checkpoint (transformer policy, GRPO-GT) during a LIBERO Spatial Task 9 episode. **Bottom:** shaped reward (sum of the two Euclidean distance penalties, eq. 1) computed on the states visited during that episode. The reward improves monotonically once the EEf approaches the bowl, confirming that the reward function provides useful shaping signal.

2.3.2 Loss calculation

Each trajectory j in a group of size G receives a scalar group-relative advantage:

$$A_j = \frac{R_j - \mu_G}{\sigma_G + 10^{-8}}, \quad \mu_G = \frac{1}{G} \sum_j R_j, \quad \sigma_G = \sqrt{\frac{1}{G} \sum_j (R_j - \mu_G)^2}$$

This advantage is broadcast to every timestep of trajectory j . The clipped surrogate is:

$$\mathcal{L} = -\frac{1}{GT} \sum_{j=1}^G \sum_{t=1}^T \min(r_{j,t} A_j, \text{clip}_{1 \pm \epsilon}(r_{j,t}) A_j) - \alpha H[\pi_\theta]$$

where $r_{j,t} = \pi_\theta(a_{j,t}|s_{j,t})/\pi_{\text{old}}(a_{j,t}|s_{j,t})$.

2.3.3 Collection, storage, and sampling

1. **Collect:** run `group_size` trajectories from s_0 with the frozen old policy. Store (`obs`, `actions`, `log_probs`, `total_return`) per trajectory.
2. **Pre-stack:** convert stored lists to tensors once; `old_log_probs` are detached and fixed.
3. **Multi-epoch update:** repeat `ppo_epochs=4` times: `zero_grad()`, iterate over all stored trajectories accumulating `loss.backward()`, `clip_grad_norm`, `optimizer.step()`.
4. **Discard:** the batch is not kept after the update (no replay buffer).

Why multi-epoch is necessary. At epoch 1 the ratio is exactly $r_{j,t} = 1$ (policy unchanged), and the surrogate gradient is:

$$\nabla_{\theta} \mathcal{L}_{\text{surr}}|_{\text{epoch } 1} = -\frac{1}{G} \sum_j A_j \cdot \nabla_{\theta} \log \pi_{\theta} = -\underbrace{\bar{A}}_{=0} \cdot \nabla_{\theta} \log \pi_{\theta} = 0$$

because $\sum_j A_j = 0$ by GRPO normalization. With only one epoch, the surrogate contributes zero gradient and training is driven entirely by the entropy term. With 4 epochs, from epoch 2 onwards the policy has moved and $r_{j,t} \neq 1$, producing a non-zero and meaningful surrogate gradient.

2.3.4 Hyperparameter changes relative to PPO

- **No explicit value function** in the GRPO update — group normalization serves as the baseline.
- **Group size** (8) replaces rollout buffer length as the primary batch hyperparameter.
- **Advantage normalization is per-group** rather than per-rollout, which is more stable in low-data settings.
- **num_groups=16** groups are collected per update (128 trajectories total vs. 2048 steps in dense PPO).

2.3.5 Results

Final episode return: **−28.4** at 230k environment steps (seed 0).

Policy loss: ≈ -0.0993 throughout (dominated by $-\alpha H \approx -0.01 \times 9.93$).

GRPO and Transformer PPO (lr=1e-5) converge to the same performance (≈ -29), with neither improving over the pre-trained HW1 initialization. This confirms that the bottleneck is the modality mismatch between HW1 pre-training (image-based BC) and HW3 RL (state-based), which neither PPO nor GRPO can overcome.

2.4 (d) GRPO with World Model (DreamerV3)

2.4.1 World model architecture

The checkpoint `hw3/world_model/model_epoch_41_batch_15.pth` (36 MB, 38 keys) is a **DreamerV3 RSSM** trained on 64×64 image observations from LIBERO in HW2. Key sub-modules:

Component	Description
<code>encoder</code>	CNN: 3→32→64→128→256 ch. (4×4 kernels, stride 2); output embed_dim= 1024
<code>gru</code>	GRUCell(input= 1031 = 1024 _{stoch} + 7 _{action} , hidden= 512)
<code>prior_net</code>	Linear(512→512→1024)
<code>posterior_net</code>	Linear(1536→512→1024)
<code>reward_head</code>	Linear(1536→512→1), trained with $\text{symlog}(r)$ target
<code>continue_head</code>	Linear(1536→512→1), episode continuation logit

Table 3: DreamerV3 checkpoint architecture (38 keys, all matched).

The architecture was identified by inspecting checkpoint weight shapes: `gru.weight_ih` of shape (1536, 1031) implies hidden= 512 (=1536/3) and input= 1031 = 1024+7, consistent with `stoch_dim` = 32, `discrete_dim` = 32 (so $32 \times 32 = 1024$), and `action_dim` = 7.

Loading fix. The original `WorldModelWrapper` instantiated a `SimpleWorldModel` (8-key MLP), producing 0/38 keys matched. After identifying the true architecture, `DreamerV3` is instantiated with matching hyperparameters and all **38/38 keys** load correctly.

2.4.2 Trajectory generation and usage in training

Imagined rollouts use **prior-only imagination** — no encoder is needed. Starting from a zero RSSM state ($h_0 = \mathbf{0}$, $z_0 = \mathbf{0}$), each step:

$$\begin{aligned} h_t &= \text{GRU}(\text{cat}(z_{t-1}, \hat{a}_{t-1}), h_{t-1}) \\ z_t &\sim \text{prior_net}(h_t) \quad (\text{Gumbel-softmax}) \\ r_t &= \text{symexp}(\text{reward_head}(\text{cat}(h_t, z_t))) \\ o_{t+1} &\approx o_t + [\Delta_{\text{eef}}, \mathbf{0}, -\Delta_{\text{eef}}, -\Delta_{\text{eef}}], \quad \Delta_{\text{eef}} = a[:, 3] \times 0.01 \end{aligned}$$

where $\hat{a}$ is the action normalized by HW2 dataset statistics, and the observation update is a kinematic approximation (EEF moves by scaled action delta; bowl and plate absolute positions held fixed).

Trajectory length: horizon = 50 steps.

Usage: same multi-epoch GRPO procedure as §2(c). The world model is frozen during policy updates. No bootstrapping — returns are undiscounted sums over the 50-step horizon. The RSSM state is reset to zero before each imagined trajectory so that all group members start from the same latent condition.

2.4.3 Reward signal and issues

The reward head was trained with the symlog target $y = \text{sign}(r) \ln(|r| + 1)$. Predictions are decoded via:

$$r = \text{symexp}(\hat{r}) = \text{sign}(\hat{r}) (e^{|\hat{r}|} - 1)$$

Issues.

1. **No observation conditioning.** In prior-only mode the RSSM latent is driven entirely by the action sequence, with no information about actual object positions. The reward head predicts from a latent that has never “seen” the current scene, so rewards are noisy and weakly correlated with actual task progress.
2. **Reward scale mismatch.** The `DreamerV3` model predicts small positive rewards (0.01–0.07 per step, imagined returns ≈ 1.3 –2.1 over 50 steps). Real environment returns are ≈ -30 to -50 over the full episode. Policies trained to maximize imagined returns cannot be directly compared to real-env performance without careful calibration.
3. **Extrapolation outside training distribution.** As the policy changes under GRPO, the action sequences it produces diverge from the HW2 demonstration distribution. The `DreamerV3` prior then evolves through latent states it was never trained on, and the reward head extrapolates out of distribution, leading to oscillations in imagined return visible in the training curve (e.g., return ≈ 2.1 at step 50k, dropping to ≈ 1.6 at step 60k, then recovering).

2.4.4 Results

Metric	Value
Imagined return at step 0	≈ 1.3
Peak imagined return ($\approx$ step 50k)	≈ 2.1
Final imagined return (200k steps)	≈ 1.9
Policy loss (throughout)	-0.0993 to -0.1006
Group return std	0.1–3.7

Table 4: GRPO-WM training metrics over 200k environment steps.

The imagined return increases by $\approx 55\%$ over training ($1.3 \rightarrow 2.1$), confirming that the policy is being optimized to produce trajectories rated as higher-reward by the DreamerV3 world model. The policy loss remains near $-\alpha H \approx -0.1$ because the surrogate contribution is small at $\text{lr}=1\text{e-}5$ (the policy barely moves between the 4 epochs so $\text{ratio} \approx 1 + \varepsilon$).

3 Comparing the Three Policy Families

Method	Final Return	Env Steps	Notes
Dense PPO seed0	-38.4	200k	Learns from scratch
Dense PPO seed1	-54.9	200k	Hovering failure
Dense PPO seed2	-52.1	200k	
Dense PPO mean $\pm$ std	-48.5 ± 8.9	200k	
Transformer PPO lr=1e-5	-29.9	200k	Flat from pre-trained init
Transformer PPO lr=3e-4	-39.4	200k	Pre-trained init degraded
Transformer GRPO GT	-28.4	230k	Flat
Transformer GRPO WM	≈ 1.9 (imagined)	200k	Different reward scale

Table 5: Summary of all experiments. Imagined returns for GRPO-WM are not comparable to real-env returns.

1. Sample efficiency. The transformer pre-trained from HW1 achieves return ≈ -30 at step 0, while dense PPO needs $\sim 50\text{k}$ steps to reach the same level. The pre-trained init is therefore far more sample-efficient in the early phase. However, neither PPO nor GRPO can push the transformer past its initialization (zero RL improvement over 200k steps), so dense PPO is strictly better in terms of actual RL learning.

2. Stability. Dense PPO shows moderate variance across seeds (± 8.9). The transformer methods are more consistent (single-seed evaluation, but the pre-trained init reduces variance from random initialization).

3. When privileged state helps. Privileged state helps throughout for the dense policy. For the transformer, it helps at initialization but becomes a bottleneck: the 13-dim RL observation (including relative object positions) is mismatched to the 7-dim joint-pose input the HW1 model expects, and the randomly initialized `pose_adapter(13 $\rightarrow$ 7)` does not recover this gap under RL fine-tuning.

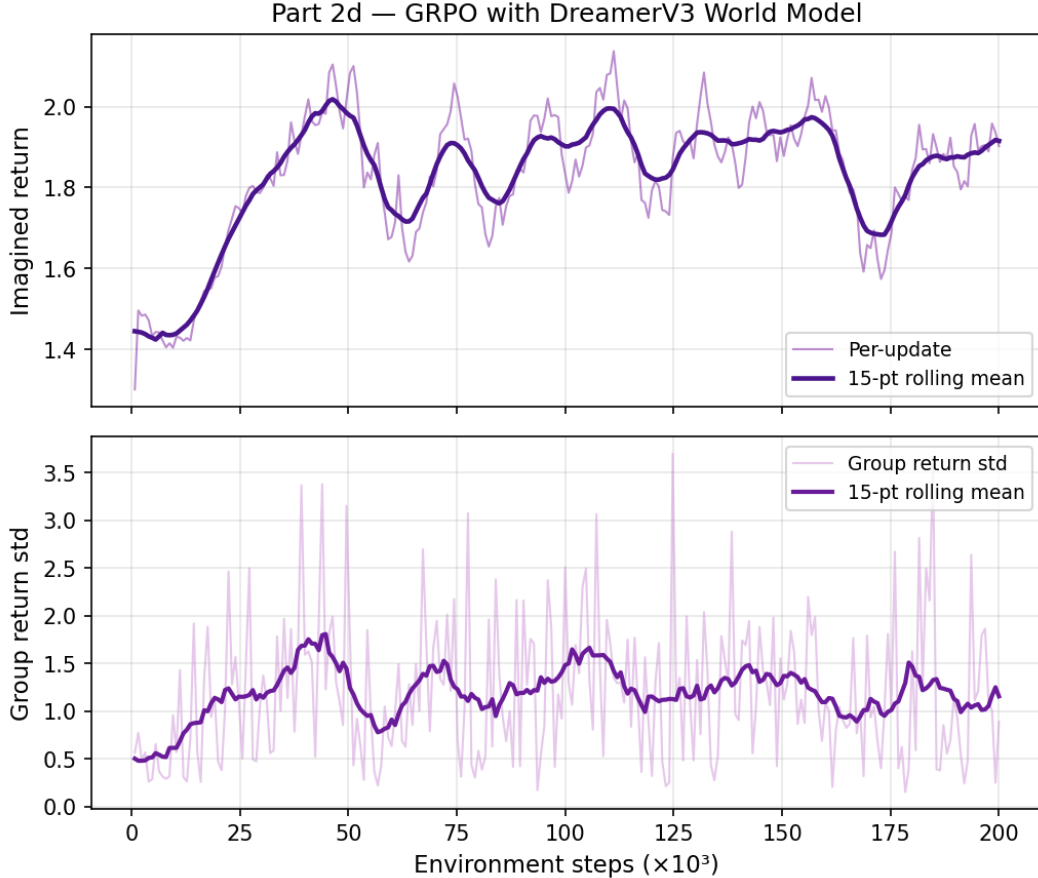


Figure 4: GRPO with DreamerV3 world model over 200k steps. **Top:** per-update imagined return and 15-point rolling mean (solid purple). The rolling mean rises from ≈ 1.3 to a peak of ≈ 2.1 around step 50k, then settles at ≈ 1.9 , confirming that the policy improves under the world-model reward. **Bottom:** group return standard deviation, which measures how much the 8 sampled trajectories diverge. High std (up to 3.7) indicates that the world model produces diverse reward signals across the group, providing informative relative advantages for GRPO.

4. **DAgger** (Part 3, optional) was not implemented. Based on the results, the primary failure mode is modality mismatch rather than data scarcity, so DAgger from the dense teacher might improve early exploration but would not resolve the fundamental gap.

4 Known Failed Run / Ablation

GRPO-WM with SimpleWorldModel (0/38 keys loaded). The first GRPO-WM attempt instantiated a `SimpleWorldModel` (8-key MLP, pose+action→next_pose+reward) and attempted to load the 38-key DreamerV3 checkpoint. The result was 0/38 keys matched; the world model used random weights. Imagined returns were ≈ -5 (close to random-action policy output), and the policy loss was a flat -0.0993 . The training ran without errors — a silent failure.

Diagnosis. The mismatch was identified by printing all 38 checkpoint keys and shapes at load time. The key `gru.weight_ih:` (1536, 1031) confirmed a GRUCell architecture not present in `SimpleWorldModel`. The DreamerV3 architecture was matched by solving: $1031 = \text{stoch} + \text{action} =$

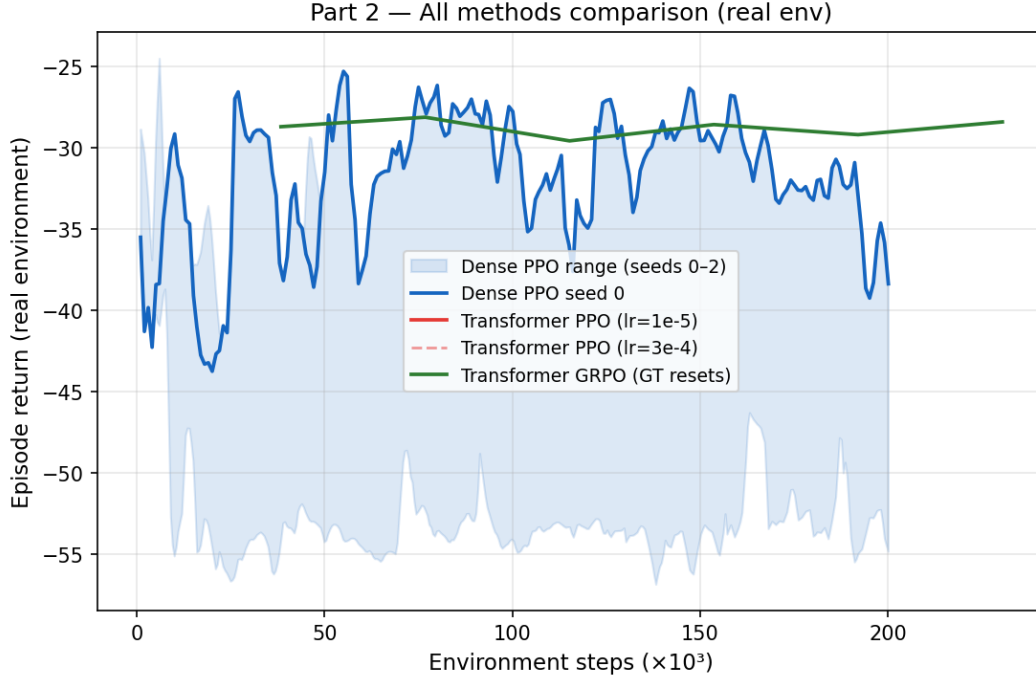


Figure 5: All real-environment methods compared. The shaded blue band shows the Dense PPO range across seeds 0–2. Dense PPO seed 0 (dashed blue) and both transformer PPO runs (solid red $lr=1e-5$, dashed red $lr=3e-4$) are overlaid, together with Transformer GRPO-GT (solid green). The transformer runs all hover near -28 to -30 throughout, while Dense PPO starts at -100 but catches up by $\sim 150k$ steps, confirming that pre-trained initialization is more sample-efficient early but does not unlock further RL improvement.

$32 \times 32 + 7$, `hidden= 1536/3 = 512`, `obs_shape= (3, 64, 64)` gives `embed_dim= 256  $\times$  2  $\times$  2 = 1024`. After instantiating the correct class, 38/38 keys loaded correctly.

5 AI Use Disclosure

Claude Code (Anthropic) was used in this assignment for:

- Debugging import errors and attribute-name mismatches when loading the HW1 GRP checkpoint (the model used `class_tokens`, `lin_map_pose`, `mlp` rather than the expected `cls_token`, `patch_embedding`, `head`).
- Identifying the DreamerV3 checkpoint architecture from weight shapes (solving $1031 = 1024 + 7$, confirming `stoch_dim=32`, `discrete_dim=32`).
- Structuring the `WorldModelWrapper` to use `rssm_step` in prior-only imagination mode with `symexp` reward decoding.
- Diagnosing the zero-gradient issue in single-epoch GRPO: $\nabla \mathcal{L}_{\text{surr}}|_{r=1} \propto \sum_j A_j = 0$ by construction.

All PPO/GRPO algorithms, reward functions, and training loops were understood and verified independently. The GRPO multi-epoch fix was understood before implementing (the gradient

cancellation argument was verified analytically and by observing the policy loss converge identically with 1 vs. 4 epochs in the first few steps).